

1. Pruebas de Muestras (MuestrasTest.java)

Descripción

Las pruebas de muestras verifican el funcionamiento del servicio CasoClinicoMuestraService de forma aislada, utilizando Mockito para simular el comportamiento del repositorio CasoClinicoMuestraRepository y ValidacionRepository.

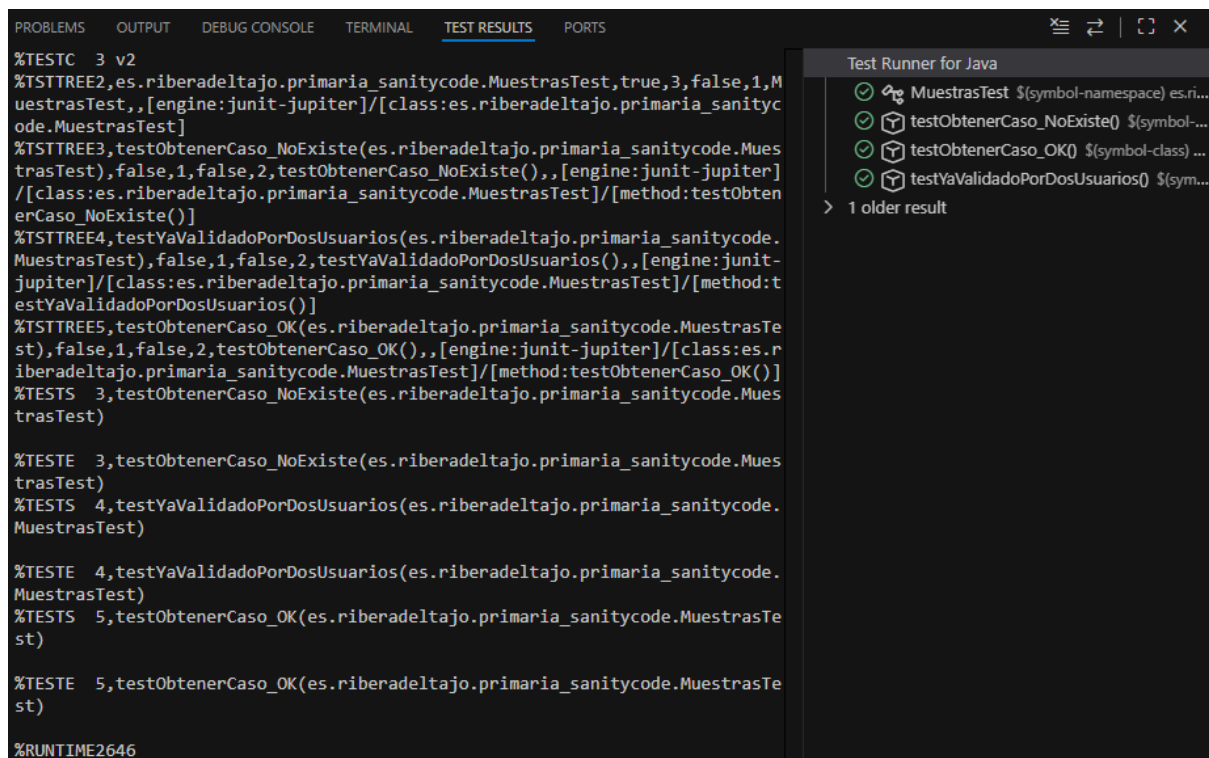
No se conecta a la base de datos real.

Configuración

- Se utiliza `@ExtendWith(MockitoExtension.class)` para habilitar Mockito.
- `@Mock` simula el repositorio.
- `@InjectMocks` inyecta el mock en el servicio.
- `@ActiveProfiles("test")` para entorno de pruebas.

Tests implementados

Muestra en Test Result:



```
%TESTC 3 v2
%TSTTREE2,es.riberadeltajo.primaria_sanitycode.MuestrasTest,true,3,false,1,MuestrasTest,,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.MuestrasTest]
%TSTTREE3,testObtenerCaso_NoExiste(es.riberadeltajo.primaria_sanitycode.MuestrasTest),false,1,false,2,testObtenerCaso_NoExiste(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.MuestrasTest]/[method:testObtenerCaso_NoExiste()]
%TSTTREE4,testYaValidadoPorDosUsuarios(es.riberadeltajo.primaria_sanitycode.MuestrasTest),false,1,false,2,testYaValidadoPorDosUsuarios(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.MuestrasTest]/[method:testYaValidadoPorDosUsuarios()]
%TSTTREE5,testObtenerCaso_OK(es.riberadeltajo.primaria_sanitycode.MuestrasTest),false,1,false,2,testObtenerCaso_OK(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.MuestrasTest]/[method:testObtenerCaso_OK()]
%TESTS 3,testObtenerCaso_NoExiste(es.riberadeltajo.primaria_sanitycode.MuestrasTest)

%TESTE 3,testObtenerCaso_NoExiste(es.riberadeltajo.primaria_sanitycode.MuestrasTest)
%TESTS 4,testYaValidadoPorDosUsuarios(es.riberadeltajo.primaria_sanitycode.MuestrasTest)

%TESTE 4,testYaValidadoPorDosUsuarios(es.riberadeltajo.primaria_sanitycode.MuestrasTest)
%TESTS 5,testObtenerCaso_OK(es.riberadeltajo.primaria_sanitycode.MuestrasTest)

%TESTE 5,testObtenerCaso_OK(es.riberadeltajo.primaria_sanitycode.MuestrasTest)

%RUNTIME2646
```

Test Runner for Java

- ✓ MuestrasTest \$(symbol-namespace) es.ri...
- ✓ testObtenerCaso_NoExiste() \$(symbol-...
- ✓ testObtenerCaso_OK() \$(symbol-class) ...
- ✓ testYaValidadoPorDosUsuarios() \$(sym...

> 1 older result

testObtenerCaso_OK

Simula que el repositorio devuelve un caso clínico válido.

Verifica que:

El resultado no es nulo.

El caso original está presente.

Muestra por consola:

```
=== INICIO TEST MUESTRAS CASOS ===
Test: obtener caso clínico correcto
20:53:30.234 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- >>> INICIO SELECCIÓN CASO | usuario=1
20:53:30.234 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- Total casos en muestra: 0
20:53:30.234 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- 📄 CASO ANALIZADO | usuario=1 | casoOriginal=1 | médicos=0
20:53:30.236 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- ✅ SELECCIONADO | usuario=1 | caso=1
Resultado: 1
Edad: 40
Diagnóstico: Migraña
✓ TEST OK
```

Objetivo: comprobar que el servicio devuelve correctamente los datos.

testObtenerCaso_NoExiste

Simula que el repositorio devuelve null.

Verifica que:

El servicio también devuelve null.

Muestra por consola:

```
=== INICIO TEST MUESTRAS CASOS ===
Test: caso clínico no existe
20:53:30.057 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- >>> INICIO SELECCIÓN CASO | usuario=1
20:53:30.071 [main] INFO es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- Total casos en muestra: 0
20:53:30.090 [main] WARN es.riberadeltajo.primaria_sanitycode.service.CasoClinicoMuestraService -- ⚠️ No hay casos disponibles | usuario=1
Resultado: null
✓ TEST OK (resultado null esperado)
```

Objetivo: comprobar el comportamiento cuando no hay datos disponibles.

testYaValidadoPorDosUsuarios

Simula que el caso clínico ya ha sido validado por dos usuarios.

Verifica que:

El servicio devuelve null.

Muestra por consola:

```
=== INICIO TEST MUESTRAS CASOS ===

Test: caso ya validado por 2 usuarios
20:53:30.189 [main] INFO es.riberadelta.primaria_sanitarycode.service.CasoClinicoMuestraService -- >>> INICIO SELECCIÓN CASO | usuario=1
20:53:30.189 [main] INFO es.riberadelta.primaria_sanitarycode.service.CasoClinicoMuestraService -- Total casos en muestra: 0
20:53:30.190 [main] INFO es.riberadelta.primaria_sanitarycode.service.CasoClinicoMuestraService -- 🚩 CASO ANALIZADO | usuario=1 | casoOriginal=1 | médicos=2
20:53:30.191 [main] INFO es.riberadelta.primaria_sanitarycode.service.CasoClinicoMuestraService -- ❌ RECHAZADO | motivo=Ya tiene 2 médicos | casoOriginal=1
Resultado: null
✓ TEST OK
```

Objetivo: comprobar que el sistema no devuelve casos ya suficientemente validados.

2. Pruebas Integración (Integraciones.java)

Descripción

Las pruebas de integración verifican el funcionamiento completo del sistema, incluyendo:

- Servicio (CasoClinicoMuestraService)
- Repositorio
- Conexión a base de datos (MariaDB)

Se utiliza `@SpringBootTest` para cargar lo utilizado de Spring.

Configuración

`@SpringBootTest` levanta la aplicación.

`@Autowired` inyecta el servicio.

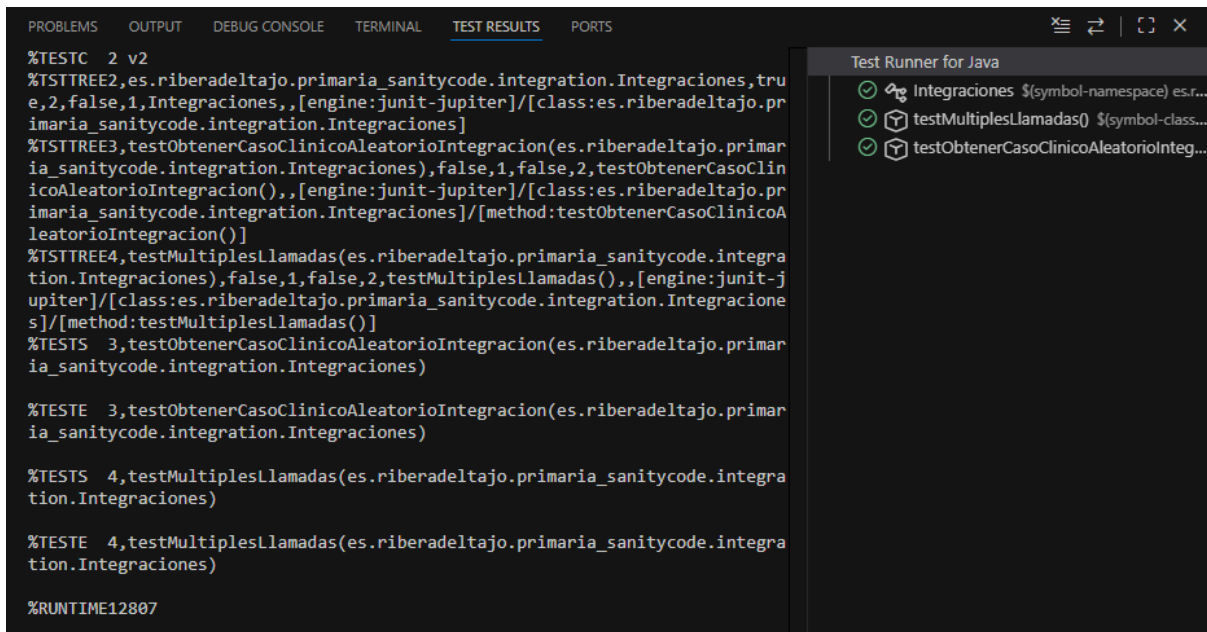
Se trabaja con datos reales (solo lectura):

- `@AutoConfigureTestDatabase(replace = NONE)` usa la BD real..

`@ActiveProfiles("test")` para entorno de pruebas.

Tests implementados

Muestra en Test Result:



testObtenerCasoClinicoAleatorioIntegracion

Obtiene un caso clínico real desde la base de datos.

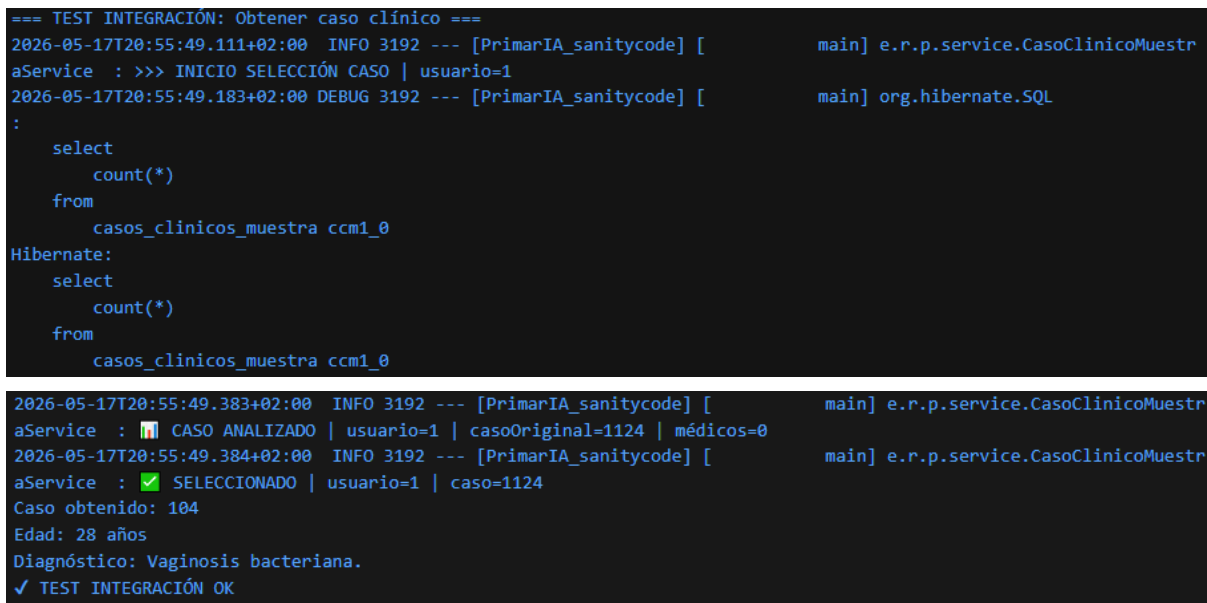
Verifica que:

- El resultado no es nulo.

- Tiene un caso original asociado.

- Contiene edad y diagnóstico.

Muestra por consola:



Objetivo: comprobar que todo el flujo (BD → repositorio → servicio) funciona correctamente.

testMultiplesLlamadas

Ejecuta varias llamadas al servicio.

Verifica que:

Ninguna devuelve null.

Muestra por consola:

```
=== TEST INTEGRACIÓN: múltiples llamadas ===
2026-05-17T20:55:49.404+02:00 INFO 3192 --- [PrimariaSanitycode] [main] e.r.p.service.CasoClinicoMuestr
aService : >>> INICIO SELECCIÓN CASO | usuario=1
2026-05-17T20:55:49.408+02:00 DEBUG 3192 --- [PrimariaSanitycode] [main] org.hibernate.SQL
:
    select
      count(*)
    from
      casos_clinicos_muestra ccm1_0
Hibernate:
    select
      count(*)
    from
      casos_clinicos_muestra ccm1_0
2026-05-17T20:55:49.608+02:00 INFO 3192 --- [PrimariaSanitycode] [main] e.r.p.service.CasoClinicoMuestr
aService : 🚩 CASO ANALIZADO | usuario=1 | casoOriginal=352 | médicos=0
2026-05-17T20:55:49.608+02:00 INFO 3192 --- [PrimariaSanitycode] [main] e.r.p.service.CasoClinicoMuestr
aService : ✅ SELECCIONADO | usuario=1 | caso=352
Iteración 2: 61
✓ TEST MÚLTIPLES OK
```

Objetivo: comprobar estabilidad del sistema en múltiples ejecuciones.

3. Pruebas Validación (ValidacionServiceTest.java)

Descripción

Estas pruebas verifican la lógica del servicio ValidacionService, incluyendo:

- Guardado de validaciones
- Gestión de usuarios
- Validación de casos clínicos

Se utilizan mocks para aislar dependencias.

Configuración

@InjectMocks para el servicio.

@Mock para repositorios:

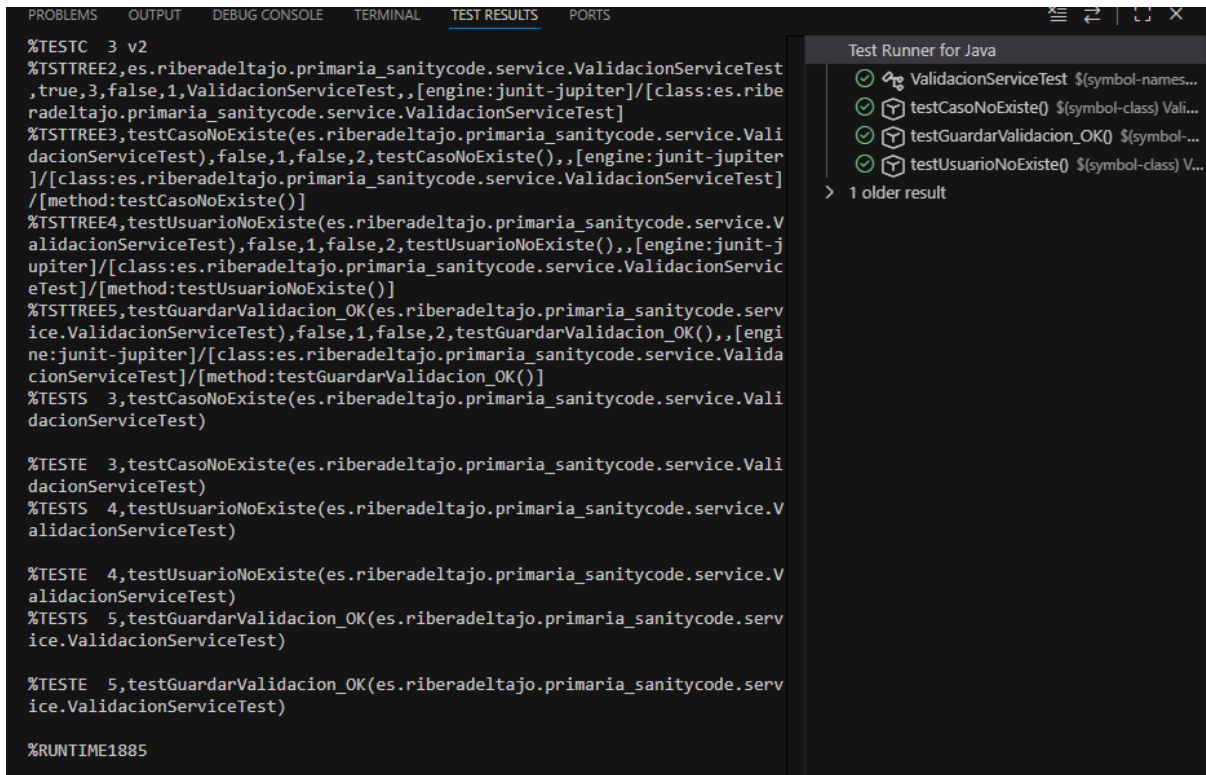
- ValidacionRepository
- UsuarioRepository
- CasoClinicoOriginalRepository

@ActiveProfiles("test") para entorno de pruebas.

MockitoAnnotations.openMocks(this) en setUp.

Tests implementados

Muestra en Test Result:



testGuardarValidacion_OK

Simula usuario y caso existente.

Verifica que:

Se guarda la validación correctamente.

Muestra por consola:

```
=== TEST VALIDACION SERVICE ===  
Test: guardar validación OK  
✓ Validación guardada correctamente
```

Objetivo: comprobar que el servicio guarda correctamente una validación cuando los datos son válidos.

testUsuarioNoExiste

Simula que el usuario no existe.

Verifica que:

Se lanza una excepción.

Muestra por consola:

```
=== TEST VALIDACION SERVICE ===  
Test: usuario no existe  
Error esperado: Usuario no encontrado  
✓ Test usuario no existe OK
```

Objetivo: comprobar que el servicio gestiona correctamente el error cuando el usuario no existe.

testCasoNoExiste

Simula que el caso clínico no existe.

Verifica que:

Se lanza una excepción.

Muestra por consola:

```
=== TEST VALIDACION SERVICE ===  
Test: caso no existe  
✓ Test caso no existe OK
```

Objetivo: comprobar que el servicio gestiona correctamente el error cuando el caso clínico no existe.

4. Prueba Repositorio (RepositoryTest.java)

Descripción

Las pruebas de repositorio verifican el acceso a la base de datos, asegurando que las consultas funcionan correctamente.

Validan que el repositorio devuelve datos reales correctamente.

Configuración

Se utiliza

@DataJpaTest para configurar el entorno de prueba con JPA y la base de datos.

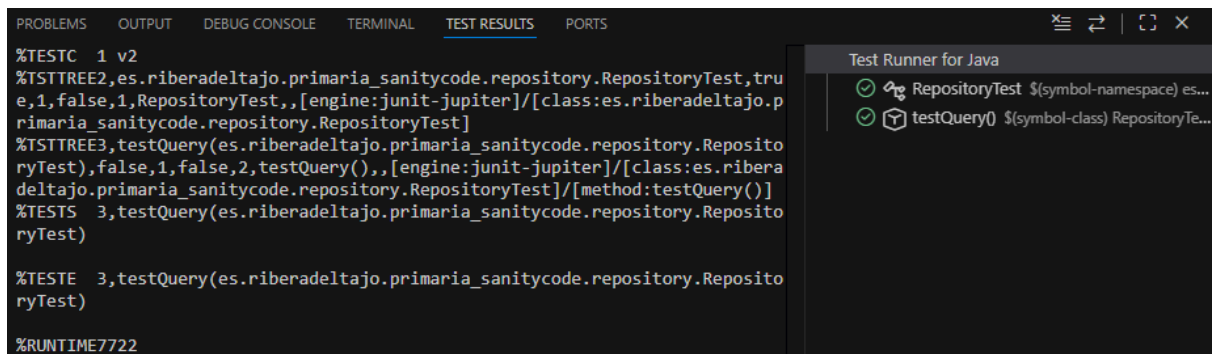
@Autowired: Inyecta el repositorio en el test.

@AutoConfigureTestDatabase(replace = NONE) usa BD real.

@ActiveProfiles("test") para entorno de pruebas.

Tests Implementados

Muestra en Test Result:



testQuery

Realiza una consulta al repositorio para obtener un caso clínico aleatorio de la base de datos.

Verifica que:

El resultado no es nulo.

La consulta retorna correctamente un caso clínico.

Muestra por consola:

```
=== TEST REPOSITORIO ===
2026-05-17T21:01:13.451+02:00 DEBUG 8500 --- [Primaria_sanitarycode] [main] org.hibernate.SQL
:
  SELECT
    m.*
  FROM
    casos_clinicos_muestra m
  WHERE
    m.id_original NOT IN (
      SELECT
        v.id_caso_original
      FROM validaciones v
      WHERE
        v.id_usuario = ?
    )
  AND (
    from
      casos_clinicos_originales cco1_0
    where
      cco1_0.id=?
  Caso obtenido: 121
  ✓ Query OK
```

Objetivo: Comprobar que la consulta findRandomCasoClinico del repositorio devuelve correctamente un caso clínico aleatoriamente desde la base de datos.

5. Pruebas OAuth2 (OAuth2SuccessHandlerTest.java)

Descripción

Estas pruebas verifican el comportamiento del login OAuth2 y la gestión de usuarios en el sistema.

Se usa Mockito para simular:

- HttpServletRequest
- HttpServletResponse
- Authentication
- OAuth2User

Configuración

Se utiliza `@ExtendWith(MockitoExtension.class)` para habilitar Mockito.

`@InjectMocks` inyecta el `OAuth2SuccessHandler`.

`@Mock` simula:

- `UsuarioRepository`
- `WhitelistRepository`

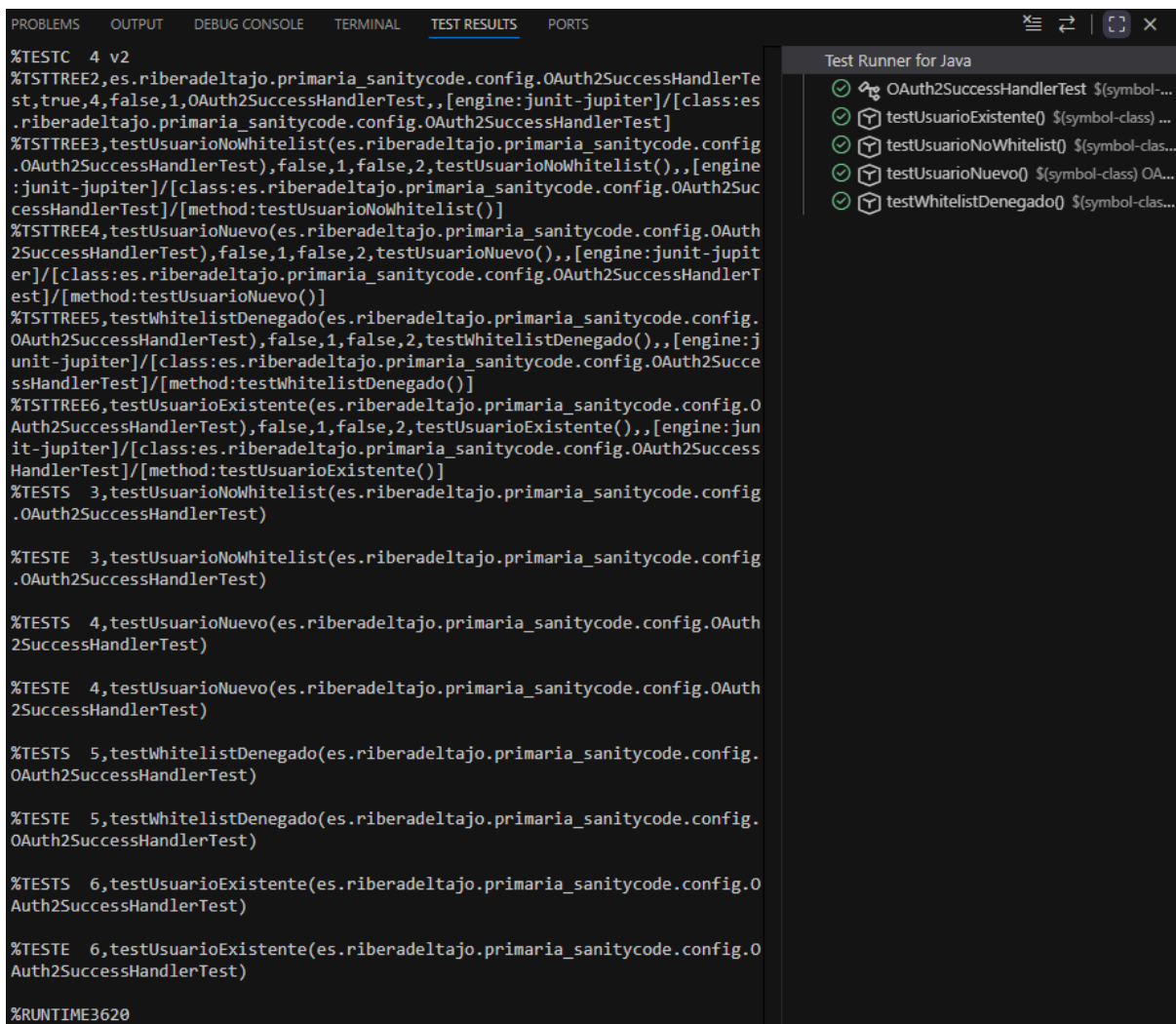
Se mockean objetos HTTP (request, response, session).

Se simula el usuario `OAuth2` mediante `OAuth2User`.

`@ActiveProfiles("test")` para entorno de pruebas.

Tests implementados

Muestra en Test Result:



```
%TESTC 4 v2
%TSTTREE2,es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest,true,4,false,1,OAuth2SuccessHandlerTest,,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest]
%TSTTREE3,testUsuarioNoWhitelist(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest),false,1,false,2,testUsuarioNoWhitelist(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest]/[method:testUsuarioNoWhitelist()]
%TSTTREE4,testUsuarioNuevo(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest),false,1,false,2,testUsuarioNuevo(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest]/[method:testUsuarioNuevo()]
%TSTTREE5,testWhitelistDenegado(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest),false,1,false,2,testWhitelistDenegado(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest]/[method:testWhitelistDenegado()]
%TSTTREE6,testUsuarioExistente(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest),false,1,false,2,testUsuarioExistente(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest]/[method:testUsuarioExistente()]
%TESTS 3,testUsuarioNoWhitelist(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTE 3,testUsuarioNoWhitelist(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTS 4,testUsuarioNuevo(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTE 4,testUsuarioNuevo(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTS 5,testWhitelistDenegado(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTE 5,testWhitelistDenegado(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTS 6,testUsuarioExistente(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%TESTE 6,testUsuarioExistente(es.riberadeltajo.primaria_sanitycode.config.OAuth2SuccessHandlerTest)

%RUNTIME3620
```

testUsuarioNoWhitelist

Simula un usuario que no está en la whitelist.

Verifica que:

La sesión se invalida.

Muestra por consola:

```
=== TEST: Whitelist no existe ===
>>> INTENTO DE LOGIN: test@test.com
>>> ACCESO DENEGADO: test@test.com no está en la whitelist.
✓ Usuario bloqueado correctamente
```

Objetivo: comprobar que el sistema bloquea usuarios no autorizados.

testWhitelistDenegado

Simula un usuario sin acceso permitido.

Verifica que:

La sesión se invalida correctamente.

Muestra por consola:

```
=== TEST: Whitelist denegado ===
>>> INTENTO DE LOGIN: test@test.com
>>> ACCESO DENEGADO: test@test.com no está en la whitelist.
✓ Sesión invalidada correctamente
```

Objetivo: comprobar que se fuerza el cierre de sesión cuando el usuario no está autorizado.

testUsuarioExistente

Simula un usuario que ya existe en el sistema.

Verifica que:

Se actualiza el usuario correctamente.

Se guarda en la base de datos.

Muestra por consola:

```
=== TEST: Usuario existente ===
>>> INTENTO DE LOGIN: test@test.com
>>> USUARIO EXISTENTE: Actualizando último acceso para test@test.com
✓ Usuario actualizado correctamente
```

Objetivo: comprobar que los usuarios existentes se actualizan correctamente tras login OAuth2.

testUsuarioNuevo

Simula un usuario nuevo que entra por primera vez.

Verifica que:

Se crea un nuevo usuario.

Se guarda en la base de datos.

Muestra por consola:

```
=== TEST: Usuario nuevo ===
>>> INTENTO DE LOGIN: new@test.com
>>> NUEVO USUARIO: Registrando new@test.com
✓ Usuario creado correctamente
```

Objetivo: comprobar que el sistema registra correctamente nuevos usuarios OAuth2.

6. Pruebas de Controladores

Descripción

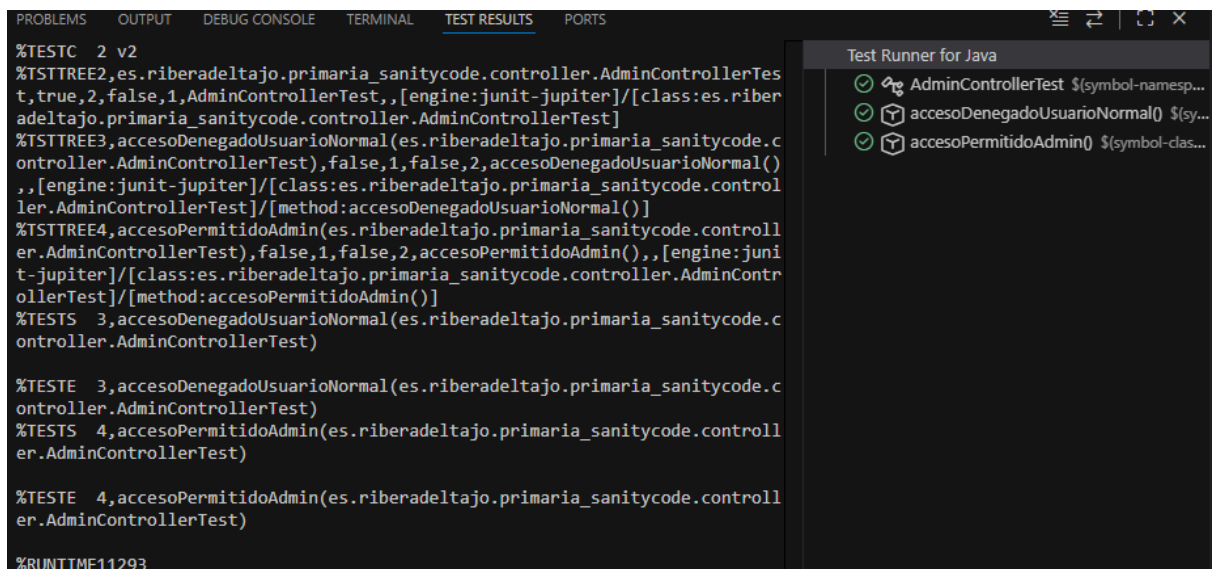
Estas pruebas verifican el comportamiento de los endpoints REST y la seguridad del sistema usando MockMvc.

Configuración

- @SpringBootTest carga el contexto.
- @AutoConfigureMockMvc permite probar endpoints.
- @ActiveProfiles("test") para entorno de pruebas.

Tests implementados(AdminControllerTest.java)

Muestra en Test Result:



```
%TESTC 2 v2
%TSTTREE2,es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest,true,2,false,1,AdminControllerTest,,[engine:junit-jupiter]/[class:es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest]
%TSTTREE3,accesoDenegadoUsuarioNormal(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest),false,1,false,2,accesoDenegadoUsuarioNormal(),,[engine:junit-jupiter]/[class:es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest]/[method:accesoDenegadoUsuarioNormal()]
%TSTTREE4,accesoPermitidoAdmin(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest),false,1,false,2,accesoPermitidoAdmin(),,[engine:junit-jupiter]/[class:es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest]/[method:accesoPermitidoAdmin()]
%TESTS 3,accesoDenegadoUsuarioNormal(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest)

%TESTE 3,accesoDenegadoUsuarioNormal(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest)
%TESTS 4,accesoPermitidoAdmin(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest)

%TESTE 4,accesoPermitidoAdmin(es.riberadeltaajo.primaria_sanitycode.controller.AdminControllerTest)

%RUNTIME11293
```

accesoDenegadoUsuarioNormal

Simula acceso con rol USER.

Verifica que:

Devuelve estado 403 Forbidden.

Muestra por consola:

```
=== TEST: Acceso denegado a usuario ===
✓ Usuario normal bloqueado correctamente
```

Objetivo: comprobar restricciones de acceso.

accesoPermitidoAdmin

Simula login OAuth2 de admin.

Verifica que:

Devuelve estado 200 OK.

Muestra por consola:

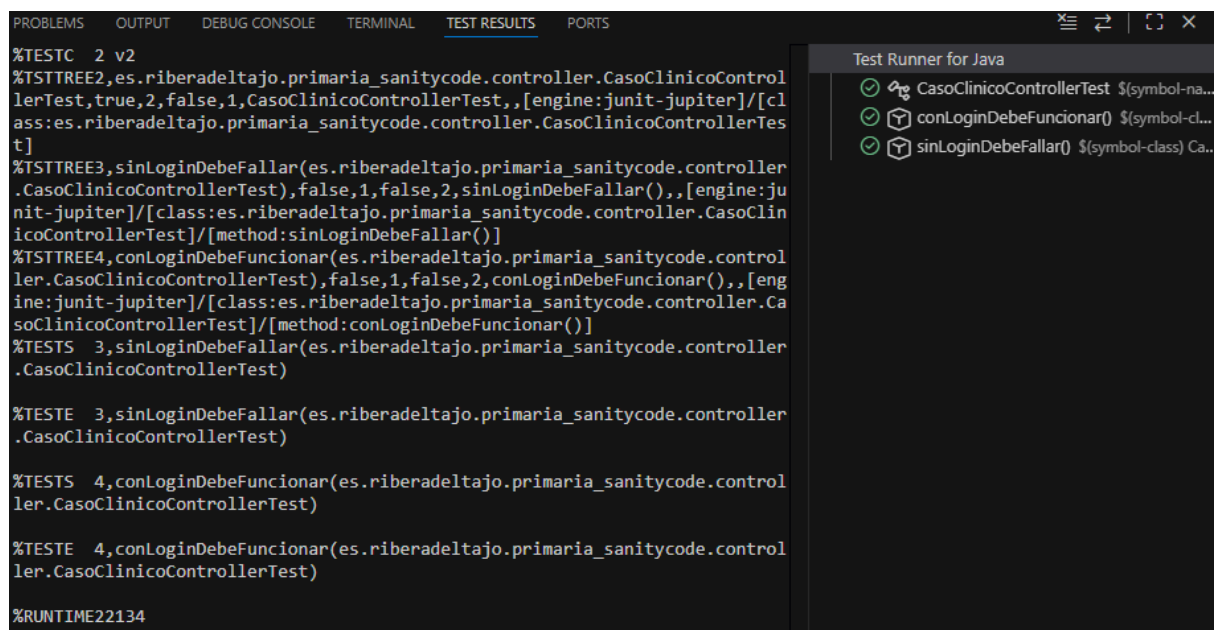
```
=== TEST: Acceso permitido a admin ===
2026-05-17T21:08:27.256+02:00 DEBUG 2380 --- [Primaria_sanitycode] [main] org.hibernate.SQL
:
    select
      u1_0.id,
      u1_0.email,
      u1_0.fecha_creacion,
      u1_0.nombre,
      u1_0.rol,
      u1_0.ultimo_acceso
    from
      usuarios u1_0
    where
      u1_0.email=?
Hibernate:
```

```
    select
      u1_0.id,
      u1_0.email,
      u1_0.fecha_creacion,
      u1_0.nombre,
      u1_0.rol,
      u1_0.ultimo_acceso
    from
      usuarios u1_0
    order by
      u1_0.fecha_creacion desc
✓ Admin accede correctamente
```

Objetivo: comprobar acceso correcto para administradores.

Tests implementados(CasoClinicoControllerTest.java)

Muestra en Test Result:



The screenshot displays the Test Results window in an IDE. The left pane shows the Test Results tab with a list of tests and their outcomes. The right pane shows the Test Runner for Java with a list of tests and their outcomes.

Test Results (Left Pane):

- %TESTC 2 v2
- %TSTTREE2,es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest,true,2,false,1,CasoClinicoControllerTest,,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest]
- %TSTTREE3,sinLoginDebeFallar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest),false,1,false,2,sinLoginDebeFallar(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest]/[method:sinLoginDebeFallar()]
- %TSTTREE4,conLoginDebeFuncionar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest),false,1,false,2,conLoginDebeFuncionar(),,[engine:junit-jupiter]/[class:es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest]/[method:conLoginDebeFuncionar()]
- %TESTS 3,sinLoginDebeFallar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest)
- %TESTE 3,sinLoginDebeFallar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest)
- %TESTS 4,conLoginDebeFuncionar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest)
- %TESTE 4,conLoginDebeFuncionar(es.riberadeltajo.primaria_sanitycode.controller.CasoClinicoControllerTest)
- %RUNTIME22134

Test Runner for Java (Right Pane):

- ✓ CasoClinicoControllerTest \$(symbol-na...
- ✓ conLoginDebeFuncionar() \$(symbol-cl...
- ✓ sinLoginDebeFallar() \$(symbol-class) Ca...

sinLoginDebeFallar

Verifica acceso sin autenticación.

Verifica que:

Devuelve estado 401 Unauthorized.

Muestra por consola:

```
=== TEST: Sin login ===  
✓ Bloqueo sin login OK
```

Objetivo: comprobar seguridad sin login.

conLoginDebeFuncionar

Verifica acceso con login OAuth2.

Verifica que:

Devuelve estado 200 OK.

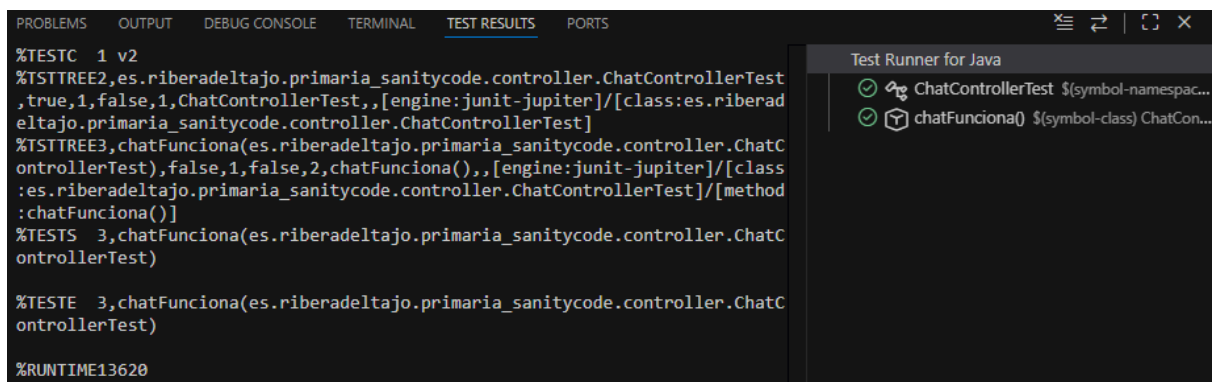
Muestra por consola:

```
=== TEST: Con login ===  
2026-05-17T21:13:57.077+02:00 INFO 9952 --- [Primaria_saniticode] [main] e.r.p.service.CasoClinicoMuestr  
aService : 🔍 Buscando usuario en BD | email=test@test.com  
2026-05-17T21:13:57.263+02:00 DEBUG 9952 --- [Primaria_saniticode] [main] org.hibernate.SQL  
:  
    select  
        count(distinct v1_0.id_usuario)  
    from  
        validaciones v1_0  
    where  
        v1_0.id_caso_original=?  
2026-05-17T21:13:57.514+02:00 INFO 9952 --- [Primaria_saniticode] [main] e.r.p.service.CasoClinicoMuestr  
aService : 📄 CASO ANALIZADO | usuario=2 | casoOriginal=3937 | médicos=0  
2026-05-17T21:13:57.514+02:00 INFO 9952 --- [Primaria_saniticode] [main] e.r.p.service.CasoClinicoMuestr  
aService : ✅ SELECCIONADO | usuario=2 | caso=3937  
✓ Acceso con login OK
```

Objetivo: comprobar acceso autenticado.

Tests implementados(ChatControllerTest.java)

Muestra en Test Result:



The screenshot shows the Test Results window in an IDE. The left pane displays the test execution details for the test class `ChatControllerTest`. The test results are as follows:

Test Class	Test Method	Test Result
<code>es.riberadeltajo.primaria_saniticode.controller.ChatControllerTest</code>	<code>chatFunciona()</code>	Passed

The right pane shows the Test Runner for Java, which displays the test results in a tree view. The test results are as follows:

- `ChatControllerTest` (symbol-namespace)
- `chatFunciona()` (symbol-class) ChatCon...

chatFunciona

Envía petición POST al chat.

Verifica que:

Devuelve 200 OK

Muestra por consola:

```
=== TEST: Chat funciona ===  
✓ Chat OK
```

Objetivo: comprobar funcionamiento del endpoint de chat.

7. Pruebas Servicio AI (AiChatServiceTest.java)

Descripción

Verifica el funcionamiento del servicio de IA.

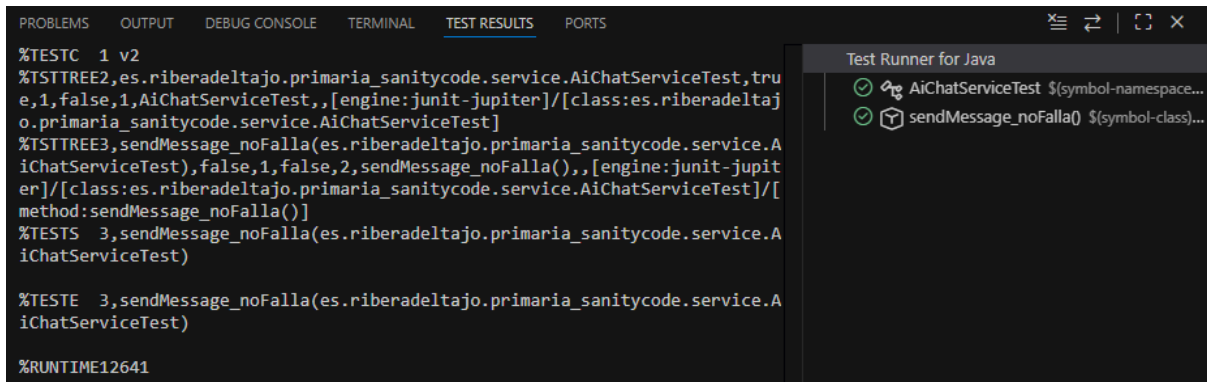
Configuración

@SpringBootTest

@Autowired para inyectar el servicio.

Tests implementados

Muestra en Test Result:



sendMessage_noFalla

Envía un mensaje al servicio.

Verifica que:

La respuesta no es null.

Muestra por consola:

```
=== TEST AI CHAT SERVICE ===  
✓ Respuesta obtenida: Hola. ¿En qué puedo ayudarle hoy?
```

Objetivo: comprobar que el servicio responde correctamente.

8. Prueba de Seguridad (SecurityTest.java)

Descripción

Comprueba que el contexto de seguridad carga correctamente.

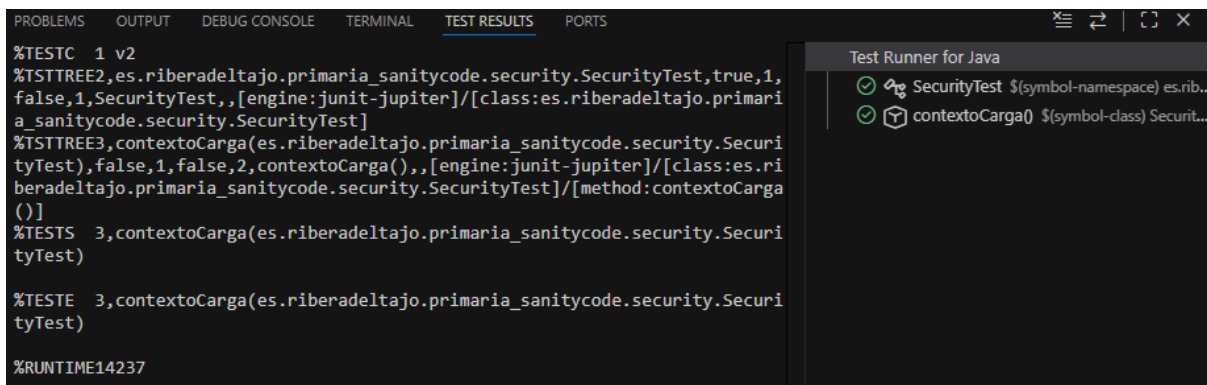
Configuración

@SpringBootTest carga el contexto completo de la aplicación.

@ActiveProfiles("test") para ejecutar en entorno de pruebas.

Tests implementados

Muestra en Test Result:



contextoCarga

Verifica que la aplicación arranca sin errores de seguridad.

Verifica que:

- El contexto de Spring Boot se carga correctamente.

- No se producen excepciones durante el arranque.

- La configuración de seguridad está correctamente inicializada.

Muestra por consola:

```
=== TEST: Carga seguridad ===  
✓ Seguridad cargada correctamente
```

Objetivo: asegurar que la configuración de seguridad es válida.

9. Conclusión

Las pruebas de muestras verifican que la lógica interna del servicio funciona correctamente mediante el uso de mocks, evitando la dependencia de la base de datos. Esto permite ejecutar pruebas más rápidas y detectar errores de forma aislada.

Las pruebas de integración verifican que el sistema funciona correctamente de extremo a extremo, asegurando la correcta comunicación entre la base de datos, el repositorio y el servicio.

Las pruebas de repositorio verifican que las consultas a la base de datos se ejecutan correctamente y que los datos se recuperan sin errores.

Las pruebas del servicio de validación verifican que la lógica de negocio funciona correctamente, tanto en el guardado de validaciones como en la gestión de errores cuando no existen usuarios o casos clínicos.

Las pruebas de controladores verifican que los endpoints REST responden correctamente, así como el cumplimiento de las restricciones de seguridad y autenticación.

Las pruebas de OAuth2 verifican que el sistema de autenticación gestiona correctamente a los usuarios, controlando el acceso mediante whitelist y asegurando la correcta creación o actualización de usuarios.

Las pruebas del servicio de IA verifican que el sistema es capaz de generar respuestas sin errores.

Las pruebas de seguridad verifican que la configuración de seguridad se carga correctamente y que la aplicación puede iniciarse sin fallos.

El uso de `System.out.println` en las pruebas permite observar los resultados directamente en la consola, facilitando la verificación visual de que las pruebas se ejecutan correctamente y entregan los resultados esperados.